

In the name of Allah, Most Gracious, Most Merciful.



CacheFlow: Memory-Aware Expert Scheduling for Efficient Mixture-of-Experts Models

Submitted By

Mir Md. Tarhimul Quader (C221017)
Md. Iftaker Ahamed Sayem (C221020)
Turja Dutta (C221026)

Supervisor

Jamil As-Ad
Lecturer,
Dept. of CSE, IIUC

Co - Supervisor

Arif Mohammed Asfe
Adjunct Lecturer,
Dept. of CSE, IIUC

Department of Computer Science and Engineering
International Islamic University Chittagong



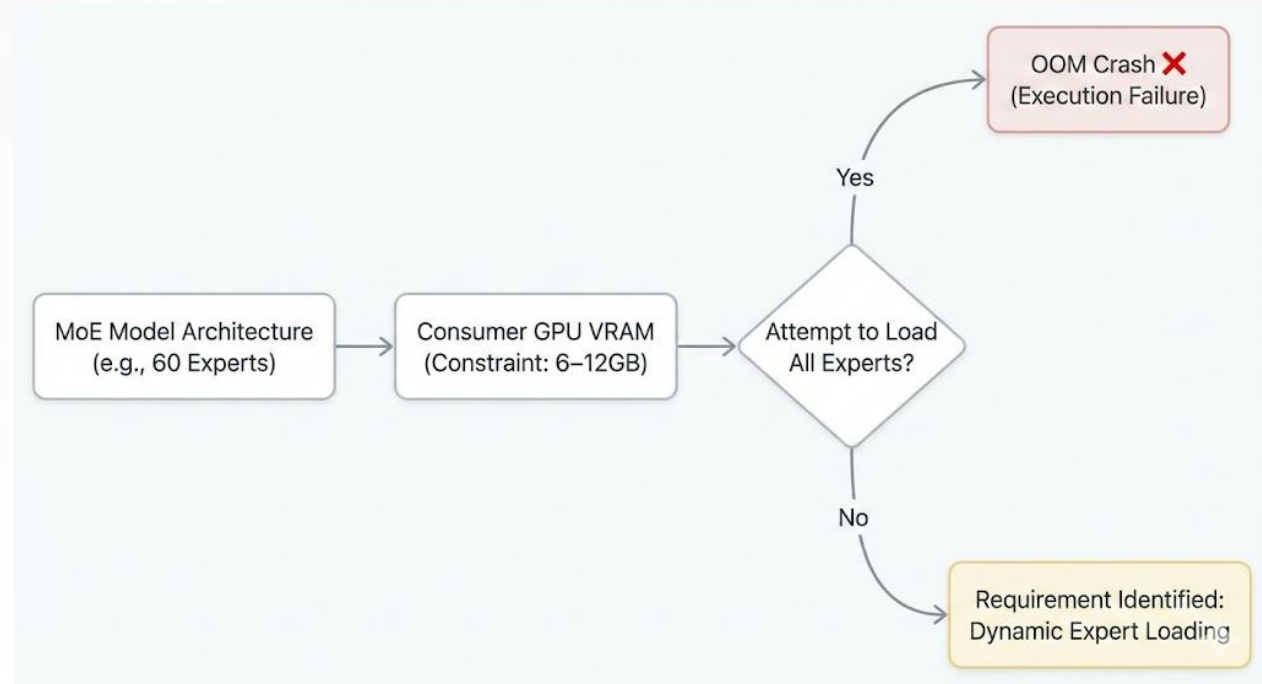
Problem Statement

- Mixture of Experts (MoE) models scale large language models by activating only a few experts per token.
- However, during inference, all Expert parameters must be loaded into GPU memory.
- This causes Out-Of-Memory (OOM) errors on consumer-grade GPUs.
- As a result, large MoE models cannot be executed in practical deployment environment.



Why Existing MoE Models Fail?

- MoE models activate few experts per token.
- But all experts are loaded into GPU memory.
- This causes Out-of-Memory (OOM) on normal GPUs.
- Therefore, large MoE models cannot run in practice.

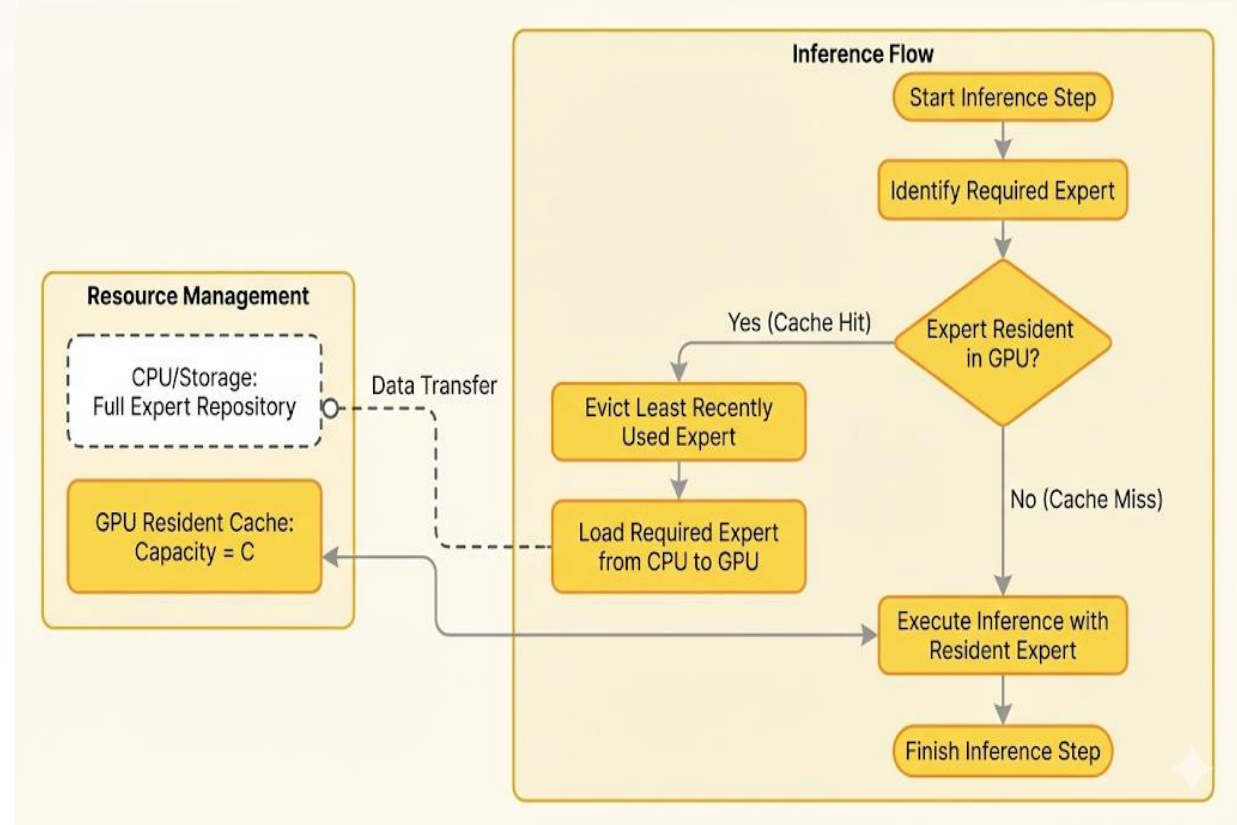


Research Gap

- Existing MoE inference frameworks assume full expert residency in GPU memory.
- Current Memory optimization techniques focus mainly on attention/KV cache, not expert parameters
- There is not effective mechanism to dynamically manage expert memory under strict VRAM constraints during inference.

Key Insight Behind CacheFlow

- Expert parameters dominate Moe memory usage.
- GPU memory is limited and fixed.
- Only a subset of experts needed at a time.
- CacheFlow dynamically manages expert residency.



Objectives

- Design a memory-aware scheduling framework for MoE inference.
- Reduce active expert residency without affecting inference correctness.
- Prevent GPU Out-of-Memory(OOM) while maintaining stable inference latency.
- Analyze the effect of cache capacity and prompt locality on system behavior.

Challenges

- **Memory Constraint:** GPU VRAM is limited and fixed.
- **Expert Thrashing:** Frequent loading and eviction can increase latency.
- **Model Opacity:** Pre-built MoE models hide expert execution logic.
- **Deployment Realism:** The solution must work with real models and real inference.

Related Work: MoE Architectures

- Mixture-of-Experts (MoE) models scale neural networks by activating only a subset of experts per token.
- This approach significantly increases model capacity without proportional computation cost.
- Modern large language models such as Switch Transformer, Mixtral, and Qwen-MoE adopt this architecture.
- Expert selection is typically performed using a gating or routing network.

Key Observation: Although computation is sparse, **expert parameters are fully loaded into GPU memory during inference.**

Related Work: Efficient LLM Inference

- Recent systems focus on optimizing inference efficiency for large language models.
- Techniques include KV-cache optimization, memory paging, and batching strategies.
- These approaches reduce attention memory overhead and improve throughput.
- Such methods are effective for dense transformer models.

Limitation: These techniques **do not address the memory footprint of MoE expert parameters.**



Related Work: MoE Memory Management

- Prior work explores expert routing efficiency and load balancing in MoE models.
- Some approaches investigate expert offloading or hierarchical execution strategies.
- However, most frameworks assume sufficient GPU memory for all experts.
- Practical deployment under strict VRAM constraints is rarely addressed.

Research Gap: There is **no unified, practical framework** that dynamically manages expert residency during real MoE inference on constrained hardware.



Baseline Framework

Baseline MoE Inference

- All experts are loaded into GPU memory before inference.
- Inference fails with OOM on constrained GPUs.
- No mechanism for expert eviction or memory prioritization.



Theoretical Background

- MoE inference involves dynamic expert selection per token.
- Only a subset of experts contributes to the output at each step.
- This sparsity creates an opportunity for memory-aware expert management.



CacheFlow Concept

- GPU memory is treated as bounded cache for expert parameters
- Only a fixed number of experts remain active at any time.
- Experts are dynamically loaded and evicted based on runtime priority.



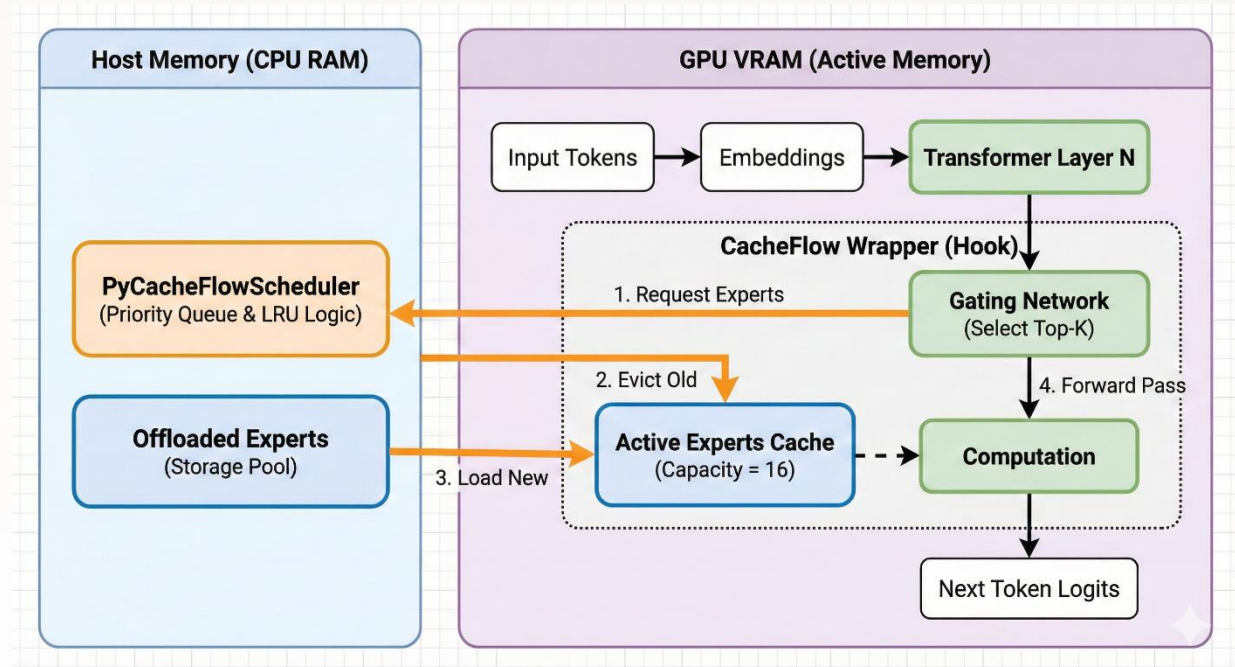
What is CacheFlow?

- CacheFlow is a memory-aware scheduling framework.
- Integrated directly into MoE inference.
- Controls which experts are loaded in GPU memory.
- Prevents OOM(Out of Memory) while preserving Inference correctness.



Methodology

- Intercept MoE execution during inference.
- Track expert usage and contextual relevance.
- Schedule expert loading and eviction dynamically.
- Preserve original model routing and outputs.



Scheduling Logic

Each expert is assigned a priority score:

$$\text{Score} = (W_{kv} \cdot \mathbb{I}_{context}) + (W_{loc} \cdot \mathbb{I}_{resident})$$

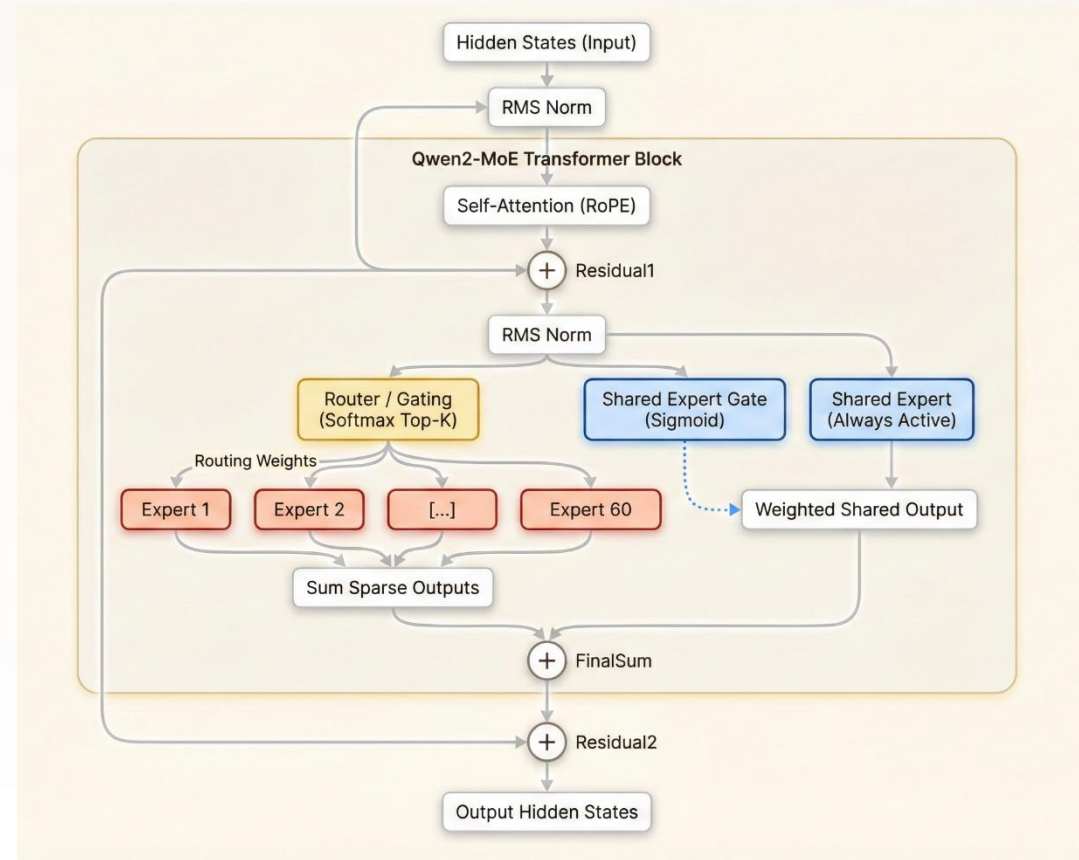
Where:

- $\mathbb{I}_{context}$ is an indicator function that takes the value of 1 when the request is related to an existing key value (KV) context, and 0 when the context does not exist.
- $\mathbb{I}_{resident}$ is an indicator function that values 1 when the requested expert is already resident in intellectual memory, and 0 when not.
- W_{kv} and W_{loc} the fixed priority weights which have values 1000 and 500 respectively in the existing implementation.

Experts with higher scores are prioritized.

Model Implementation

- The Qwen2-MoE architecture was manually reconstructed.
- Attention, MLP and Sparse MoE blocks implemented explicitly.
- CacheFlow logic injected inside the MoE forward pass.
- Ensures full control over expert execution and memory usage.



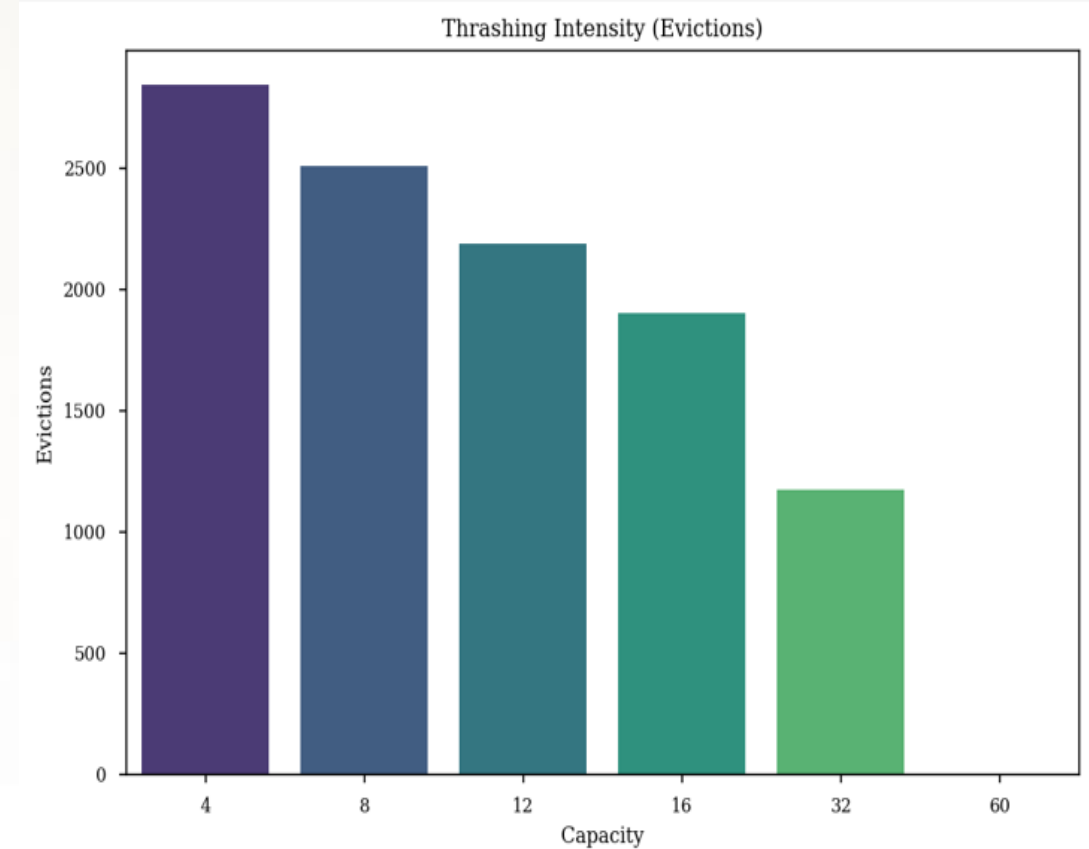
Experimental Settings

- **Programming Language:** Python.
- **Frameworks:** PyTorch, Transformers, Accelerate.
- **Quantization:** 4-bit NF4.
- **Hardware:** NVIDIA T4 GPU (~15GB VRAM).
- **Execution:** Real inference on GPU.



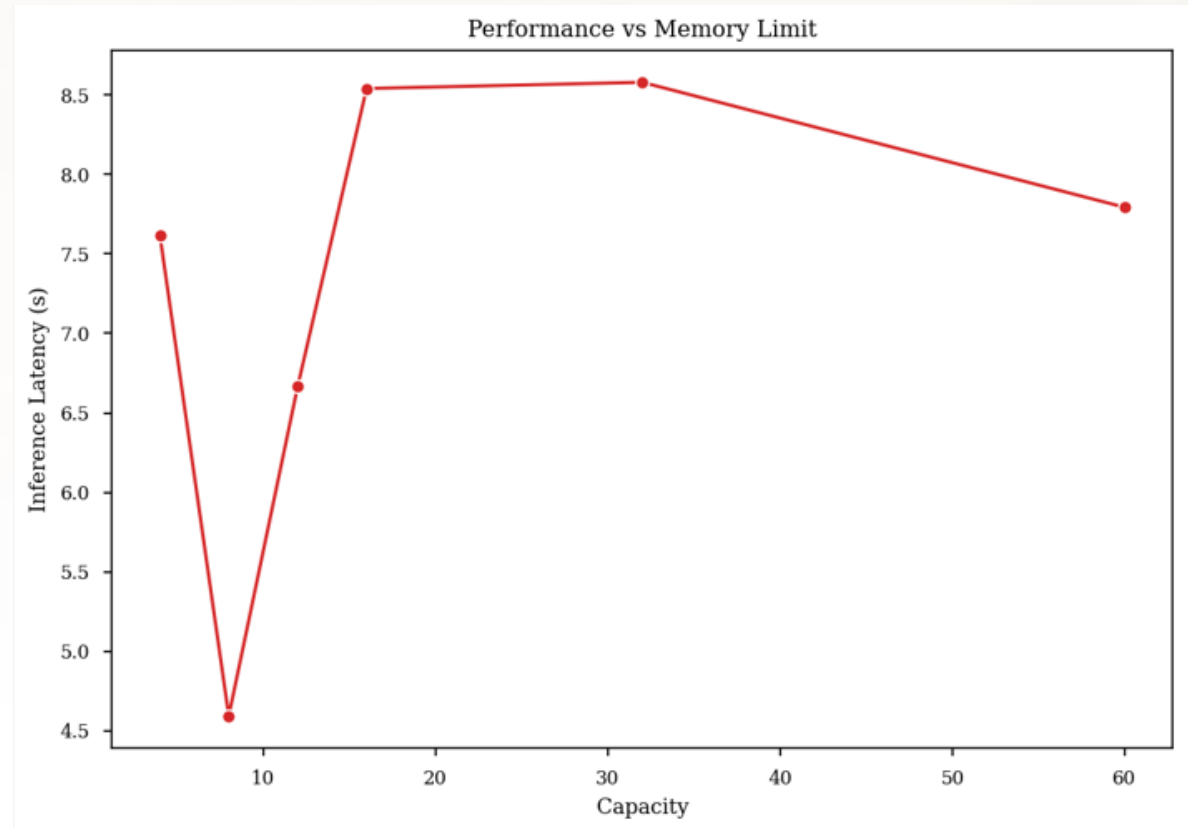
Thrashing Behavior

- Thrashing occurs when cache capacity is smaller than the expert working set.
- Frequent expert eviction increase data transfer overhead.
- Low capacity values results in high eviction counts.
- Increasing cache capacity reduces thrashing significantly.
- Stable behavior is observed once the working set fits in GPU memory



Impact of Prompt Locality

- Cache capacity was varied to observe latency behavior.
- Small cache sizes cause frequent expert swapping.
- Latency stabilizes once the working set fits in memory.



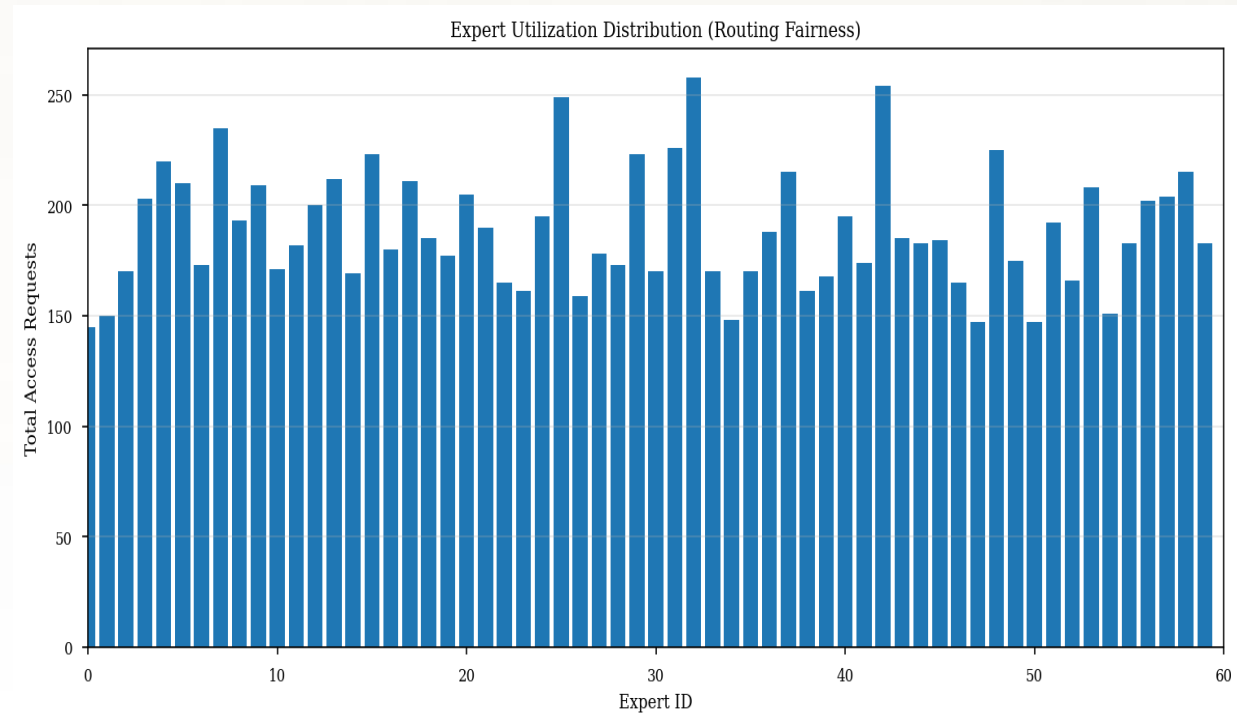
Behavioral Analysis

- Expert residency heatmap show temporal locality.
- A small subset of experts remains active across tokens.
- Confirms existence of a stable working set.



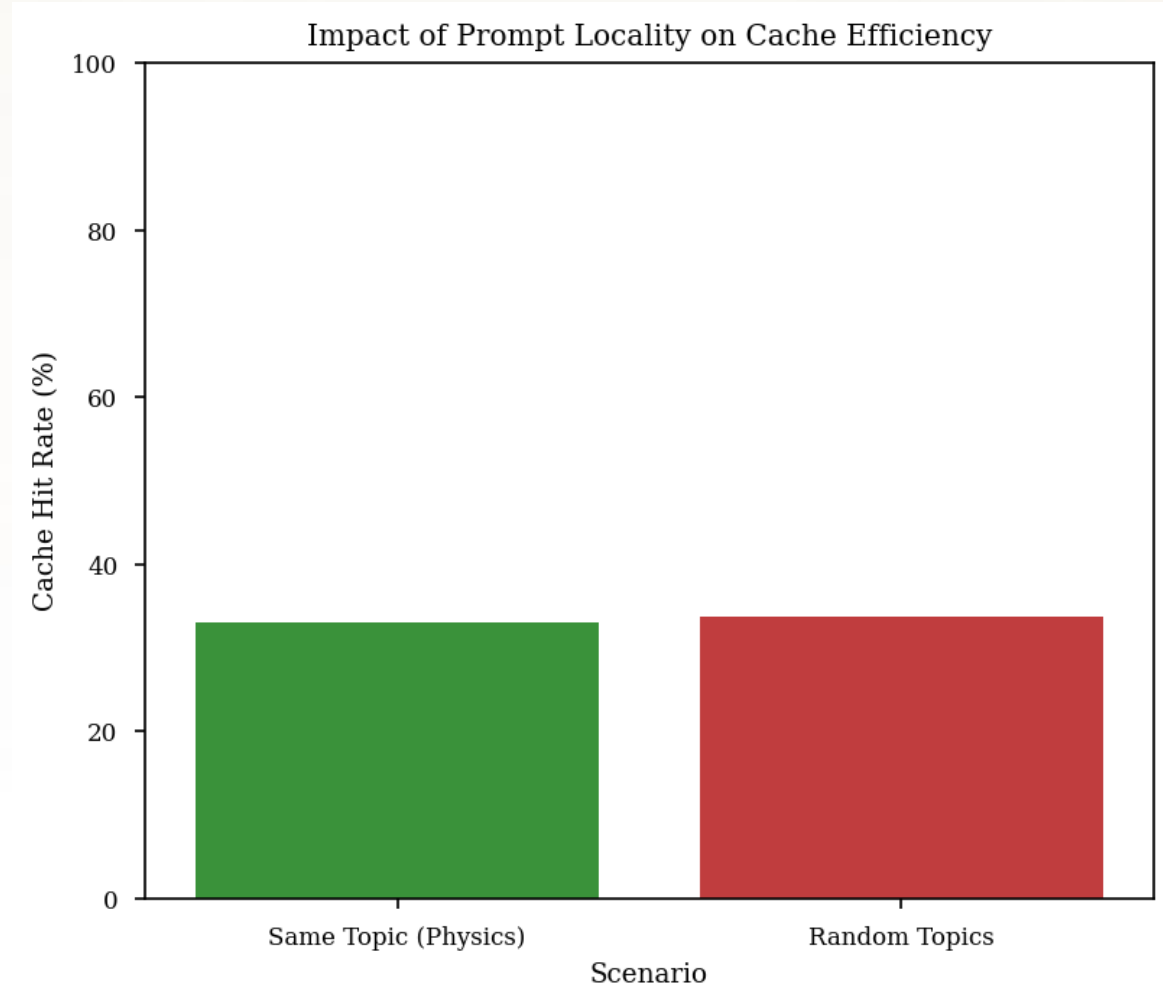
Routing Fairness

- Expert utilization remains well-distributed.
- No collapse to a single expert observed.
- CacheFlow does not interfere with routing behavior.



Impact of Prompt Locality

- Coherent-topic prompts improve cache hit rate.
- Random-topic prompts reduce reuse and increase evictions.
- Demonstrates semantic locality in MoE expert usage.



Memory Optimization Analysis

- Active experts reduced from 60 to 16 ($\approx 73\%$).
- Total VRAM usage reduced by $\approx 47\text{-}50\%$.
- Static weights remain fixed in memory.
- Enables large MoE inference where baseline fails.



Discussion

- Baseline MoE inference results in OOM.
- CacheFlow enables practical execution under memory constraints.
- Working set behavior eliminates thrashing.
- Prompt locality significantly affects system efficiency.



Conclusion

- Proposed CacheFlow, a memory-aware scheduling framework for MoE inference.
- Integrated into a custom Qwen2-MoE implementation.
- Achieved significant expert and VRAM reduction.
- Demonstrated stable and correct inference on constrained hardware.



Future Work

- Optimize scheduling logic using lower-level implementations.
- Develop predictive expert pre-fetching mechanisms.
- Extend support to other MoE architectures (Mixtral, Grok)



References

- Vaswani, A., et al. Attention Is All You Need. NeurIPS, 2017. [arXiv:1706.03762](#)
- Radford, A., et al. Improving Language Understanding by Generative Pre-Training. OpenAI, 2018. [OpenAI](#)
- Radford, A., et al. Language Models are Unsupervised Multitask Learners. OpenAI, 2019. [OpenAI](#)
- Zhang, B., Sennrich, R. Root Mean Square Layer Normalization. NeurIPS, 2019. [arXiv:1910.07467](#)
- Shazeer, N. GLU Variants Improve Transformer. arXiv preprint, 2020. [arXiv:2002.05202](#)
- Dosovitskiy, A., et al. An Image is Worth 16x16 Words. ICLR, 2020. [arXiv:2010.11929](#)
- Fedus, W., et al. Switch Transformers. JMLR, 2021. [arXiv:2101.03961](#)
- Touvron, H., et al. LLaMA: Open and Efficient Foundation Models. arXiv, 2023. [arXiv:2302.13971](#)
- Zuo, S., et al. MoE-Infinity. arXiv, 2023. [arXiv:2305.11794](#)
- Yoo, H., Lee, H. SwapMoE. arXiv, 2023. [arXiv:2309.09515](#)
- Kwon, W., et al. PagedAttention. SOSP, 2023. [arXiv:2309.06180](#)
- Liang, W., et al. DeepSeek-V2. arXiv, 2024. [arXiv:2405.04434](#)
- Shen, X., et al. fMoE. arXiv, 2024. [arXiv:2403.12881](#)